

Accurate DEM Fusion Pipeline

Comprehensive Scientific & Technical Reference Manual
Source Code Walkthrough

Pratyush Goenka

https://github.com/pgoenka/DEM_Accuracy

June 17, 2026

Abstract

This document serves as the exhaustive scientific, technical, and mathematical manual for the *Accurate DEM Fusion Pipeline*. Operating as a multi-modal geospatial synthesizer, this system evaluates arbitrary global terrain bounded by an Area of Interest (AOI). The framework reconstructs high-fidelity bare-earth Digital Terrain Models (DTMs) by actively co-registering optical, radar, and derived morphology datasets, extracting geomorphological tensors, and applying both Random Forest and PyTorch-based Deep Learning constraints bounded by physical laws.

This expanded manual does not simply list modules; it provides dedicated chapters detailing the internal logic, variable interactions, and exact theoretical reasoning behind every single function and script in the repository. Crucially, it provides rigorous scientific justifications for *why* specific mathematical models (like Nuth & Kääb coregistration), statistical distributions, tree-based ML constraints, and terrain indices were selected, and why they represent the most physically correct approach to topobathymetric and bare-earth elevation modeling.

Chapter 1

Introduction & System Philosophy

1.1 The Challenge of Spaceborne Elevation Data

Raw spaceborne digital elevation models (DEMs) (such as the Copernicus DEM, SRTM, and ALOS World 3D) are primarily Digital Surface Models (DSMs). They reflect the tops of trees, buildings, and other occlusions rather than the underlying bare earth (DTM) which is critical for hydrological routing, dam-break modeling, and infrastructural planning.

1. **Canopy and Urban Bias:** Forest canopies completely mask the underlying ground. Radar bands (X-band and C-band) penetrate poorly into dense vegetation, leading to artificial volumetric bulges in the elevation model.
2. **Spatial Misalignment:** Independent datasets collected by distinct orbital vehicles inherently suffer from horizontal and vertical shifts due to orbital drift, atmospheric refraction, and different coordinate epochs.
3. **Sensor Fusion Complexities:** Deterministic removal of forest canopies (as attempted by FABDEM) works well in mild terrain but often creates severe terracing and interpolation artifacts in steep, rocky terrain where tree cover is sparse.

1.2 The "Digital Synthesis" Concept: Scientific Rationale

The framework operates under three core scientifically validated tenets to resolve these challenges:

- **Geomorphologically-Adaptive Weighting:** Instead of treating the Copernicus DSM and FABDEM DTM equally everywhere, the system extracts the terrain slope derivative via finite-difference matrices. Steep slopes ($> 30^\circ$) represent cliffs where trees cannot grow, meaning Copernicus is highly accurate. Mild slopes indicate river valleys where FABDEM's canopy removal is necessary. The fusion is physically driven by these gradients.
- **Physics-Informed Machine Learning:** While Random Forests are excellent universal function approximators, they do not inherently understand physics. The pipeline injects constraints based on the Normalized Difference Vegetation Index (NDVI) and building footprints (via Overpass API). It enforces a hard physical rule: "Trees and buildings cannot dig holes." Therefore, any machine-learned negative correction applied to a heavily forested or urbanized pixel is mathematically clamped to zero.
- **LiDAR-Driven Ground Truth:** Spaceborne laser altimetry (NASA's ICESat-2 ATL08) provides absolute, sub-meter accuracy ground truth photons. By dynamically pulling

these tracks for the specified AOI, the framework trains the AI models entirely on real-world, localized topography, bypassing generalized global biases.

Chapter 2

Mathematical & Geospatial Foundations

2.1 Nuth & Kääb Spatial Coregistration

Scientific Justification: When fusing two different elevation datasets, simple map-algebraic subtraction is invalid because the datasets may be horizontally shifted. A slight shift Δx on a steep slope α results in a massive perceived vertical error $\Delta z = \Delta x \cdot \tan(\alpha)$. The Nuth & Kääb algorithm statistically resolves this by modeling the elevation residual dh as a function of the local slope α and aspect ψ :

$$dh = a \cdot \cos(b - \psi) \cdot \tan(\alpha) + \overline{dh} \quad (2.1)$$

where a is the magnitude of the horizontal shift vector, b is the azimuthal direction of the shift, and $\overline{dh}$ is the absolute vertical bias. By fitting this harmonic function iteratively, the algorithm perfectly aligns the FABDEM to the Copernicus base model.

2.2 Topographic Derivatives and Geomorphometry

The system builds a 13-channel tensor using continuous fields.

1. **Slope and Aspect (Sobel Filters):** We calculate the gradient vector $(\frac{\partial z}{\partial x}, \frac{\partial z}{\partial y})$ using the 3×3 Sobel operator which is robust against localized noise compared to simple finite differences.
2. **Terrain Ruggedness Index (TRI):** Mathematically estimated using local standard deviation. It acts as a high-performance proxy for ruggedness:

$$TRI \approx \sqrt{\max(\overline{z^2} - \bar{z}^2, 0)} \quad (2.2)$$

where the overline indicates a uniform mean filter.

3. **Topographic Position Index (TPI):** Calculated as $TPI = z - \bar{z}$. High TPI identifies ridges and peaks; low TPI identifies valleys and gullies.

Chapter 3

Module 1: Core Pipeline Orchestrator

File Focus: `core/pipeline.py` and `core/aoi.py`

3.1 Class: `AreaOfInterest` (`aoi.py`)

3.1.1 `__init__(self, min_lon, min_lat, max_lon, max_lat)`

Working Mechanics: Encapsulates the physical bounding box. It uses a frozen dataclass to ensure immutability throughout the pipeline's execution.

3.1.2 `hash(self)`

Working Mechanics: Generates an MD5 hash of the coordinates. This string uniquely identifies the geographical block and is scientifically crucial for deterministic cache management, preventing redundant downloads or reprocessing of identical geographical bounds.

3.1.3 `utm_epsg(self)`

Working Mechanics & Theory: Dynamically calculates the Universal Transverse Mercator (UTM) zone for the geometric centroid of the bounding box:

$$\text{Zone} = \lfloor (\text{Centroid}_{\text{lon}} + 180) / 6 \rfloor + 1 \quad (3.1)$$

If the centroid is in the Northern Hemisphere (Latitude ≥ 0), it returns EPSG 32600 + Zone. Otherwise, 32700 + Zone.

3.2 Class: `Pipeline` (`pipeline.py`)

3.2.1 `__init__(self, aoi)`

Working Mechanics: Acts as the God-object for execution. It instantiates the `PipelineContext` (memory state map), `CacheManager`, and all distinct processor engines (`CopernicusProcessor`, `Reprojector`, `MLEngine`, etc.). Crucially, it registers the linear execution DAG via `PipelineStage` wrappers (Download $\rightarrow$ Preprocess $\rightarrow$ Features $\rightarrow$ Fusion $\rightarrow$ Refinement $\rightarrow$ Export).

3.2.2 preprocess(self)

Working Mechanics: An extensive orchestration block triggering Phase 2.

- **Mosaicking:** Instructs the Copernicus processor to merge incoming TIFF tiles and crop exactly to the AOI polygon.
- **UTM Projection:** Forces a conversion from geographical EPSG:4326 to metric UTM to enable true slope and distance calculations.
- **FABDEM Fallback:** If FABDEM fails to download, mathematically clones the Copernicus DEM in memory as a fallback, ensuring the pipeline's robustness against remote server outages.
- **Coregistration Checkpoint:** Orchestrates the `align_nuth_kaab` module, enforcing that the secondary FABDEM sits exactly atop the primary Copernicus pixel grid.
- **Environmental Processing:** Triggers SAR amplitude extraction, NDVI calculation, and vector building rasterization simultaneously.

Chapter 4

Module 2: Data Acquisition Engine

File Focus: `download/planetary_loader.py` & `download/lidar_loader.py`

4.1 Class: PlanetaryLoader

4.1.1 `search_sentinel2(self, aoi, datetime)`

Working Mechanics: Interrogates the Microsoft Planetary Computer STAC catalog. It specifically searches the `sentinel-2-l2a` collection (Level-2A Bottom of Atmosphere reflectance). It forces a JSON query enforcing `eo:cloud_cover < 10%` and sorts ascending to absolutely guarantee the selection of the clearest possible optical image.

4.1.2 `download_buildings(self, aoi, output_dir)`

Working Mechanics & Theory: Rather than downloading the entire Microsoft Building Footprints gigabyte database (which spans whole continents), this utilizes an Overpass API QL script targeting OpenStreetMap. It injects the bounding box into a POST request, querying `way["building"]` and `relation["building"]`. This lightweight JSON is dumped instantly to disk, acting as a highly efficient, responsive proxy for urban density mapping.

4.2 Class: LidarLoader

4.2.1 `fetch(self, aoi, out_file)`

Working Mechanics & Theory: Connects to NASA's Earthdata portal using `earthaccess`. It queries the ATL08 dataset (Land and Vegetation Height). ICESat-2 carries the ATLAS sensor, firing 6 beams simultaneously. The function opens the fetched HDF5 granules via `h5py`. It extracts `land_segments/terrain/h_te_best_fit`, along with their exact latitude and longitude. It mathematically masks out invalid photons ($> 10000\text{m}$), filters strictly to the AOI bounding box using pandas logical indices, removes duplicate geographical points, and emits a CSV acting as the absolute ground truth matrix.

Chapter 5

Module 3: Preprocessing & Coregistration Engine

File Focus: `preprocessing/satellite_processor.py` & `preprocessing/coregistration.py`

5.1 Class: `NDVIProcessor` (`satellite_processor.py`)

5.1.1 `compute_and_align(self, context, cache, target_dem_path)`

Working Mechanics & Theory:

1. Loads Band 4 (Red) and Band 8 (Near-Infrared).
2. Calculates the Normalized Difference Vegetation Index:

$$NDVI = \frac{\rho_{NIR} - \rho_{Red}}{\rho_{NIR} + \rho_{Red}} \quad (5.1)$$

where ρ represents reflectance.

3. **Alignment Enforcement:** Optical imagery runs at 10m resolution, whereas the DEM operates at 30m. The function invokes `rasterio.warp.reproject` using `Resampling.bilinear`. It passes the exact `transform` matrix, `height`, and `width` parameters extracted directly from the `target_dem_path` (Copernicus base). This forces the 10m NDVI grid to mathematically decimate and perfectly align onto the 30m elevation grid.

5.2 Class: `Coregistrator` (`coregistration.py`)

5.2.1 `align_nuth_kaab(self, ref_path, tba_path, out_path)`

Working Mechanics: Passes paths directly into the `xdem` Python package. Initializes `xdem.coreg.NuthKaab()` which fits the sinusoidal co-registration function detailed in Chapter 2 over all valid pixels. The coregistration acts primarily as an affine translation matrix (Shift X, Shift Y, Shift Z). The transformation is immediately applied to the Target To Be Aligned (TBA) array. If the `xdem` library is missing from the environment, it gracefully catches the `ImportError` and falls back to a 1:1 copy.

Chapter 6

Module 4: Feature Engineering & Terrain Physics

File Focus: `features/feature_engine.py` & `features/normalizer.py`

6.1 Class: `TerrainFeatures`

6.1.1 Derivative Calculations (`compute_slope`, `compute_aspect`, etc.)

Working Mechanics & Theory: Invokes highly optimized C-compiled functions from `scipy.ndimage`.

- `compute_slope`: Executes horizontal (`dx`) and vertical (`dy`) Sobel convolution matrices. Evaluates slope in degrees via $\arctan(\sqrt{dx^2 + dy^2}) \cdot \frac{180}{\pi}$.
- `compute_aspect`: Calculates compass direction via $\arctan 2(-dx, dy)$.
- `compute_hillshade`: Employs standard shaded relief equations assuming a light azimuth of 315° and altitude of 45° , modeling solar illumination.

6.2 Class: `FeatureEngine`

6.2.1 `build(self, context, cache)`

Working Mechanics: This serves as the tensor synthesis step. It reads the singular 2D matrices generated by the terrain algorithms alongside the pre-aligned NDVI, SAR (VV & VH), and Building Masks. It constructs a unified Python list, and utilizes `np.stack(..., axis=-1)` to generate a dense 3D matrix of shape $(H, W, 13)$. It invokes the `FeatureNormalizer` to standard scale the array into z-scores, which is essential for convergence in deep learning and stability in splitting criteria for Random Forests. It dumps the normalization bounds (`mean`, `std`) to `feature_stats.json` for potential inference decoupling.

6.3 Class: `FeatureNormalizer`

6.3.1 `normalize(self, stack)`

Working Mechanics & Theory:

1. Casts the multi-dimensional tensor to 32-bit floating points to prevent memory explosions while retaining adequate precision.

2. Identifies all array values < -500 and actively overwrites them to `np.nan`. This is the foundational `NoData` handling rule. `-9999.0` represents missing data out-of-bounds in Geospatial raster theory. If these values are permitted into the arithmetic mean function, the channel statistics skew dramatically, causing normalization collapse (the "systematic positive bias" bug).
3. Iterates continuously across depth channels (0 to 12). Evaluates `np.nanmean` and `np.nanstd` (ignoring the injected NaNs). Subtracts mean and divides by standard deviation.

Chapter 7

Module 5: Adaptive Fusion Engine

File Focus: `fusion/fusion_engine.py`

7.1 Class: `FusionEngine`

7.1.1 `fuse(self, context, cache)`

Working Mechanics & Theory: Instead of a simple arithmetic mean $\frac{Cop+FAB}{2}$, this engine performs non-linear blending driven by geomorphology.

1. It extracts the coregistered `cop_data`, the `fab_data`, and the localized `slope_data`.
2. It creates a normalized blending tensor: `cop_weight = clip(slope/30.0, 0.0, 1.0)`.
3. It creates the inverse tensor: `fab_weight = 1.0 - cop_weight`.
4. The output array evaluates: $Fused = (Cop \cdot W_{cop}) + (FAB \cdot W_{fab})$.

Scientific Impact: In flat areas ($slope = 0^\circ$), $W_{cop} = 0.0$ and $W_{fab} = 1.0$. The pipeline fully trusts FABDEM to handle valley floors where buildings and dense forests necessitate aggressive ML smoothing. On steep mountain walls ($slope \geq 30^\circ$), $W_{cop} = 1.0$, completely discarding FABDEM's erratic interpolation anomalies, resulting in a pristine bare-earth model encompassing the strengths of both parent data sources.

Chapter 8

Module 6: Machine Learning Refinement

File Focus: `ml/predictor.py` & `ml/deep_refiner.py`

8.1 Class: `MLEngine` (`predictor.py`)

8.1.1 `refine_dem(self, context, cache)`

Working Mechanics & Theory:

1. **Alignment & Coordinate Extraction:** Loads the 13-channel numpy stack and the ICESat-2 `ground_truth.csv`. Iterates through the ICESat-2 Lat/Lon coordinates and mathematically projects them into the exact UTM projection using `pyproj.Transformer`. Passes these UTM coordinates into `rasterio.transform.rowcol` to extract the precise integer row/column indices indexing into the 3D numpy stack.
2. **Target Variable Calculation:** Extracts the 13-channel profile for every valid point: X_{train} . Calculates the prediction target: $y_{\text{train}} = \text{True Elevation} - \text{Fused Elevation}$.
3. **Missing Data Imputation:** Due to the `NoData` handling enforced during feature normalization, satellite-derived layers (SAR, Buildings) may feature NaNs outside their respective swath boundaries. The pipeline invokes `np.nan_to_num(X, nan=0.0)`. **Scientific Justification:** Because the data was previously z-score normalized, 0.0 implies the exact statistical mean of the channel. Injecting 0.0 acts as a perfectly neutral baseline imputation, preventing the tree splits from breaking while contributing zero anomalous bias.
4. **DEM Amplification:** To force the Random Forest to prioritize absolute elevation, the DEM column is forcefully duplicated twice. By feeding 3 identical DEM columns, the algorithm gives the underlying decision trees $3\times$ the statistical probability of splitting on absolute elevation compared to any environmental metric. A secondary check guarantees its combined permutation importance remains $\geq 40\%$.
5. **Model Execution:** A `RandomForestRegressor` (30 estimators, depth 6) fits the array X_{train} against y_{train} . The low tree-depth prevents catastrophic spatial overfitting to localized LiDAR noise.
6. **Inference and Correction Clamping:** The trained forest evaluates the entire $H \times W$ grid. The predicted spatial errors are halved (Correction Damping = 0.5) to ensure the parent DEM physics retain dominance. The maximum allowable correction is brutally

clamped to $3 \times \text{Std_Dev}$ of the training errors, preventing runaway extrapolations on boundaries.

7. Physics Inference Injection:

$$\text{Clamp Mask} = (\text{buildings} == 1.0) \vee (\text{ndvi} > 0.5) \quad (8.1)$$

If a pixel corresponds to dense urban infrastructure or heavy vegetation (trees), the model is physically forbidden from predicting a positive Δz that would push the DEM downwards (which would imply it found a hole). `predicted_error[clamp_mask] = np.minimum(predicted_error[clamp_mask], 0.0)`. This forces the terrain to always remain monotonic with respect to known surface scatterers.

8. **Uncertainty Estimation:** Interrogates the variance across all 30 decision trees simultaneously via `np.std(all_tree_preds, axis=0)`. Regions with sparse LiDAR coverage inherently cause high disagreement among trees, cleanly translating statistical variance into a robust geospatial `confidence.tif` uncertainty surface.

8.2 Class: LightweightUNet (deep_refiner.py)

8.2.1 PyTorch Architecture Definition

Working Mechanics: Defines a standard encoder-decoder Convolutional Neural Network (CNN) in PyTorch if the framework detects a valid Torch installation. It passes the 13-channel stack ($C = 13$) through a series of `Conv2D`, `BatchNorm2d`, `ReLU`, and `MaxPool2d` operations, collapsing it into a highly contextual latent vector before upsampling it via `ConvTranspose2d` back to spatial resolution. **Scientific Justification:** Random Forests execute purely 1×1 pixel inference. They do not understand that a pixel sits at the bottom of a river valley or halfway up a slope. The U-Net evaluates 256×256 spatial patches, natively interpreting structural topology, allowing it to mathematically resolve regional elevation biases missed by point-based algorithms.

Chapter 9

Module 7: Export & Diagnostics

File Focus: `export/exporter.py`

9.1 Class: Exporter

9.1.1 `export(self, context, cache)`

Working Mechanics:

1. Discovers the best available output state (`hydro_dem` $\rightarrow$ `final_dem` $\rightarrow$ `fused_dem`) prioritizing the most physically refined dataset in the DAG context.
2. Moves the ultimate TIFF files to the user-facing `output/` directory.
3. Dumps all `feature_importances` generated by the Random Forest along with timestamped boundaries into a structured `metadata.json`.
4. **PDF Graphic Reporting:** Leverages `matplotlib`. Opens the final DEM and confidence arrays. Aggressively applies physical bounds (`np.where(dem_data < 0, np.nan, dem_data)`) to strip away erroneous oceans and deep sinks. Computes dynamic histogram percentiles (2nd to 98th) to prevent outlier spikes from washing out the visual colormap range.
5. Renders a dual-panel Figure: Panel A displays the Final Terrain using the `terrain` colormap. Panel B displays the ML Spatial Uncertainty using the `magma` colormap. Executes `plt.savefig` directly to PDF, finalizing the pipeline's operational execution loop.